

Modeling and Verification of a
Data Aligner
using SystemVerilog and UVM

Architecture of The Verification Environment

by

Hossam Shafer Anis

Cairo, 2026

Contents

Architecture of The Verification Environment	1
Contents	2
List of Figures	3
1 Introduction.....	4
2 The Top-Level Module	4
3 The Environment.....	4
4 Agents.....	6
4.1 apb_agent.....	7
4.2 md_agent_master	8
4.3 md_agent_slave	9
5 Sequence Items	10
5.1 APB Sequence Items.....	10
5.2 MD Sequence Items	10
6 The Model.....	13
6.1 The Register Model.....	15
7 The Scoreboard	16
8 Sequences and Tests	18
8.1 Sequences	18
8.2 Tests.....	19
9 Future Improvements	20

List of Figures

Figure 1 Structure of the top-level module tb.....	4
Figure 2 Structure of the environment m_algn_env (transaction-level communication is not shown)	5
Figure 3 Structure of the environment m_algn_env (transaction-level communication is shown).....	5
Figure 4 UML class diagram of algn_env	6
Figure 5 apb_agent, md_agent_master, and md_agent_slave extend uvm_agent	6
Figure 6 Structure of m_apb_agent.....	7
Figure 7 UML class diagram of apb_agent	7
Figure 8 Structure of m_md_agent_master.....	8
Figure 9 UML class diagram of md_agent_master	8
Figure 10 Structure of m_md_agent_slave	9
Figure 11 UML class diagram of md_agent_slave	9
Figure 12 UML class diagrams of apb_seq_item_*	10
Figure 13 UML class diagrams of md_seq_item_*	11
Figure 14 Driving the Rx interface using md_seq_item_drv_master	11
Figure 15 Driving the Tx interface using md_seq_item_drv_slave	12
Figure 16 Monitoring the Rx and Tx interfaces using md_seq_item_mon.....	12
Figure 17 Block diagram of the Data Aligner module.....	13
Figure 18 Structure of model	13
Figure 19 UML class diagram of algn_model	14
Figure 20 Registers of the Data Aligner module.....	15
Figure 21 UML class diagrams of algn_reg_block and the classes of which it is composed.....	15
Figure 22 Structure of scoreboard	16
Figure 23 UML class diagram of algn_scoreboard	17
Figure 24 UML class diagrams of md_sequence_* and algn_sequence_*	18
Figure 25 UML class diagrams of algn_test_*.....	19

1 Introduction

The **Data Aligner** module takes in an unaligned stream of data and outputs it as an aligned stream of data based on its configuration.¹

In [this project](#), the Data Aligner is modeled and verified using the SystemVerilog HDVL and the Universal Verification Methodology (UVM).

This project is the course project of this Udemmy course: [Design Verification with SystemVerilog/UVM](#).

This document shows the architecture of the verification environment, the transaction-level communication between its UVM components, and UML class diagrams of some classes.

2 The Top-Level Module

[Figure 1](#) shows the structure of the top-level module, `tb`.

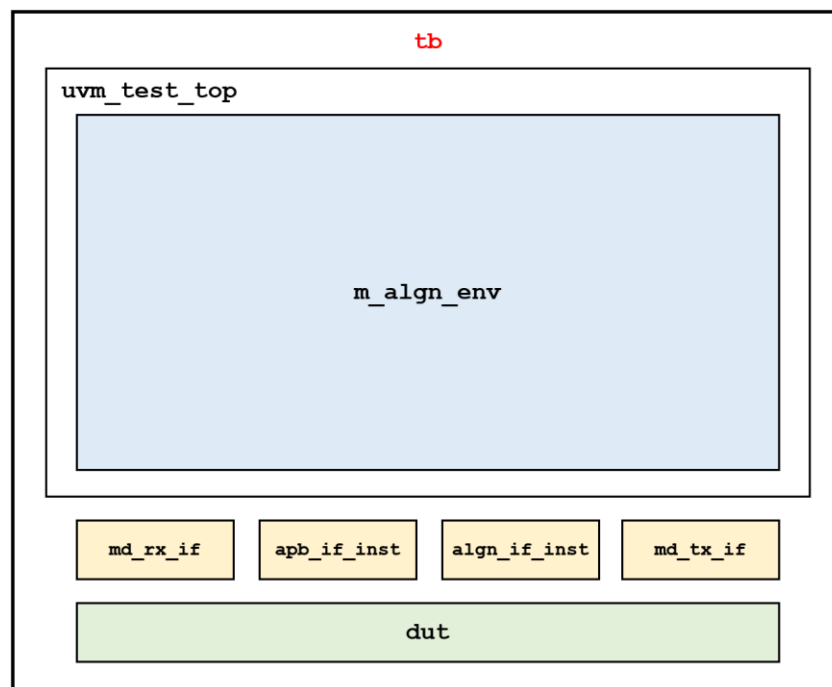


Figure 1 Structure of the top-level module `tb`

3 The Environment

[Figure 2](#) shows the structure of the environment, `m_algn_env`, which is an instance of `algn_env`. (Transaction-level communication between its sub-components is omitted for clarity).

[Figure 3](#) shows the structure of the environment with the transaction-level communication between its sub-components.²

[Figure 4](#) shows the UML class diagram of `algn_env` which extends `uvm_env`.

¹ For more information about the Data Aligner module (its interfaces, registers, functionality, and more), see its specification: *Aligner Datasheet - v1.1*.

² In this document, names of TLM ports are in **green**.

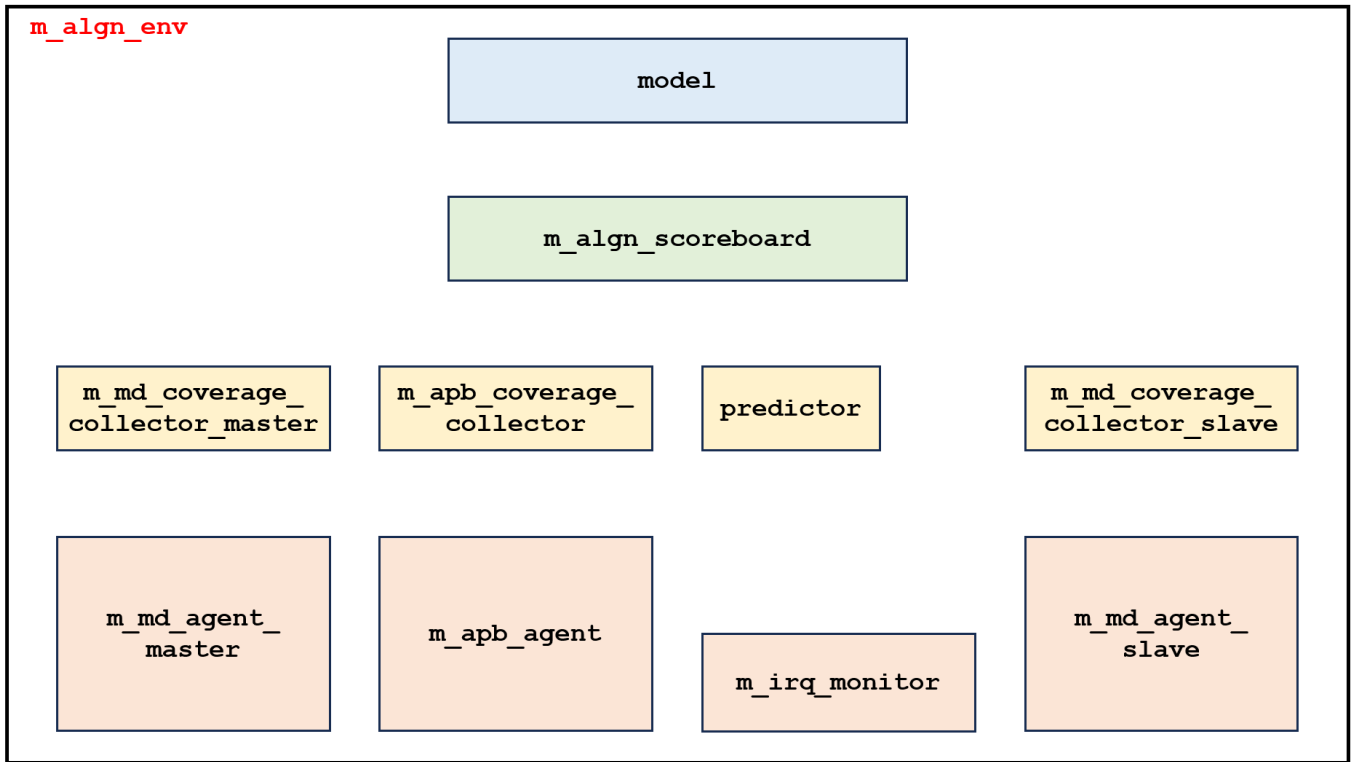


Figure 2 Structure of the environment `m_algn_env` (transaction-level communication is not shown)

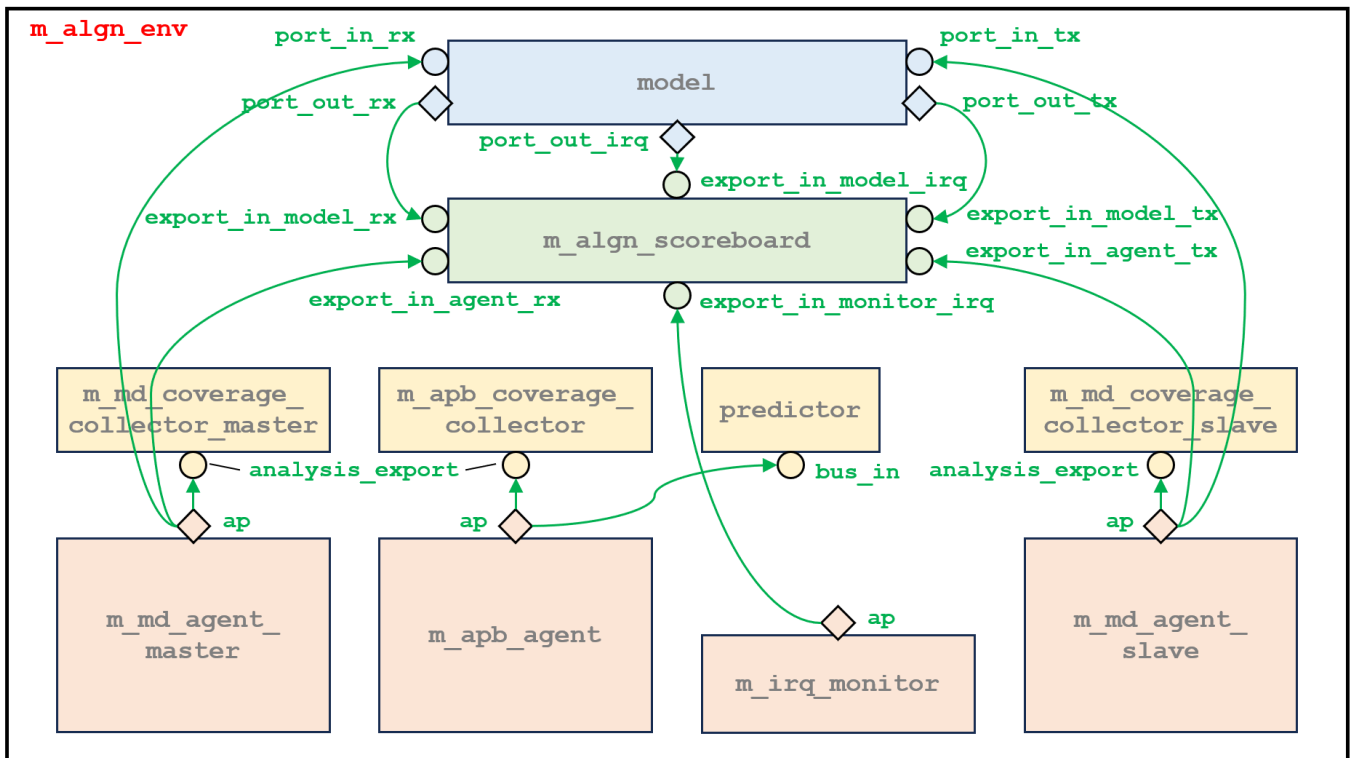


Figure 3 Structure of the environment `m_algn_env` (transaction-level communication is shown)

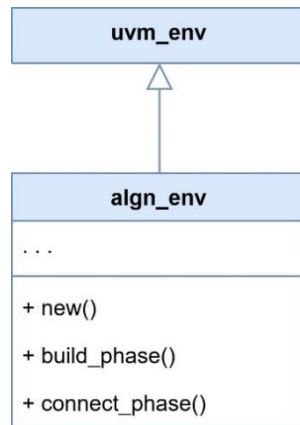


Figure 4 UML class diagram of algn_env

4 Agents

There are three agents in our environment:

- m_apb_agent, which is an instance of apb_agent.
- m_md_agent_master, which is an instance of md_agent_master.
- m_md_agent_slave, which is an instance of md_agent_slave.

These three agents extend uvm_agent. See [Figure 5](#).

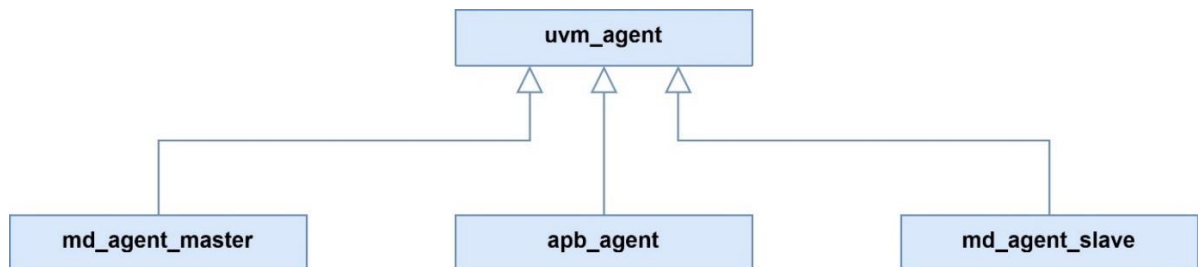


Figure 5 apb_agent, md_agent_master, and md_agent_slave extend uvm_agent

4.1 apb_agent

Figure 6 shows the structure of m_apb_agent, which is an instance of apb_agent.

Figure 7 shows the UML class diagram of apb_agent.

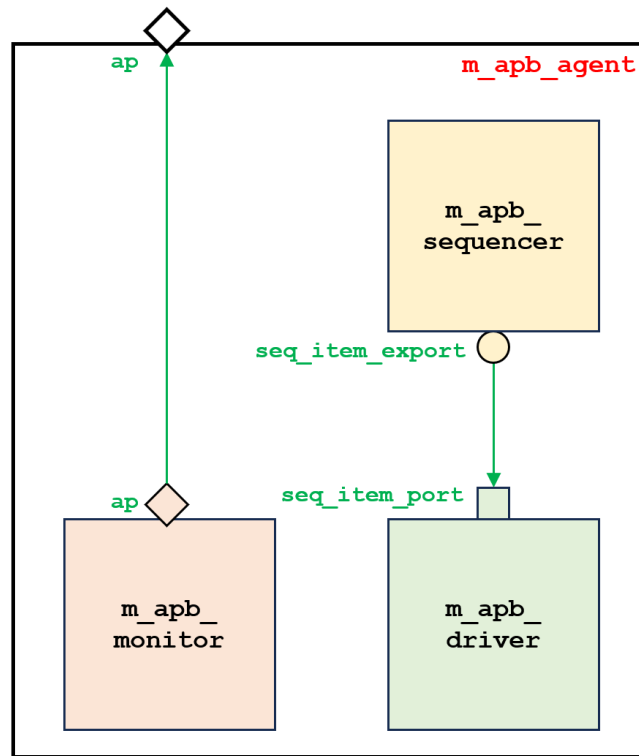


Figure 6 Structure of m_apb_agent

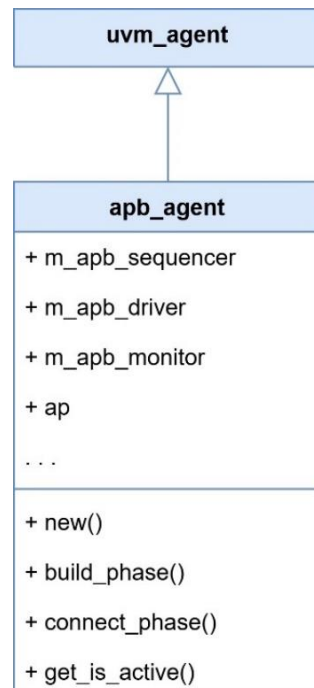


Figure 7 UML class diagram of apb_agent

4.2 md_agent_master

Figure 8 shows the structure of `m_md_agent_master`, which is an instance of `md_agent_master`.

Figure 9 shows the UML class diagram of `md_agent_master`.

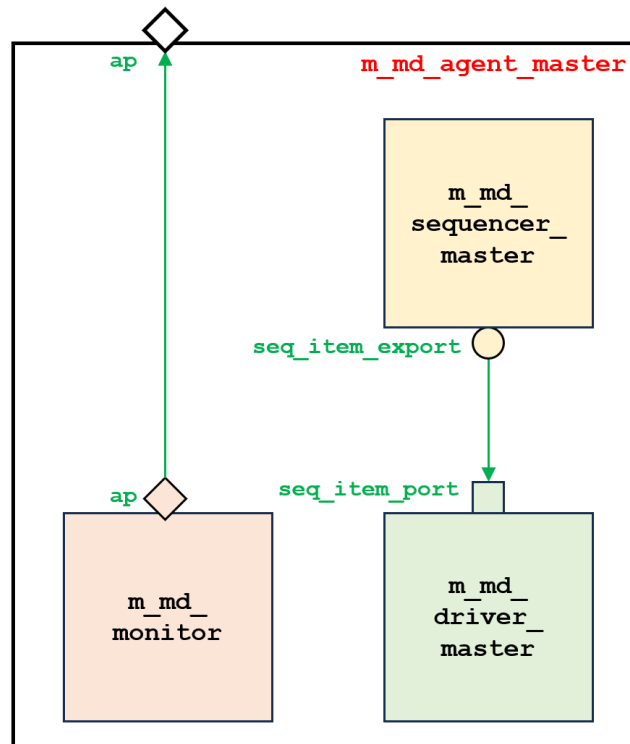


Figure 8 Structure of `m_md_agent_master`



Figure 9 UML class diagram of `md_agent_master`

4.3 md_agent_slave

Figure 10 shows the structure of m_md_agent_slave, which is an instance of md_agent_slave.

Figure 11 shows the UML class diagram of md_agent_slave.

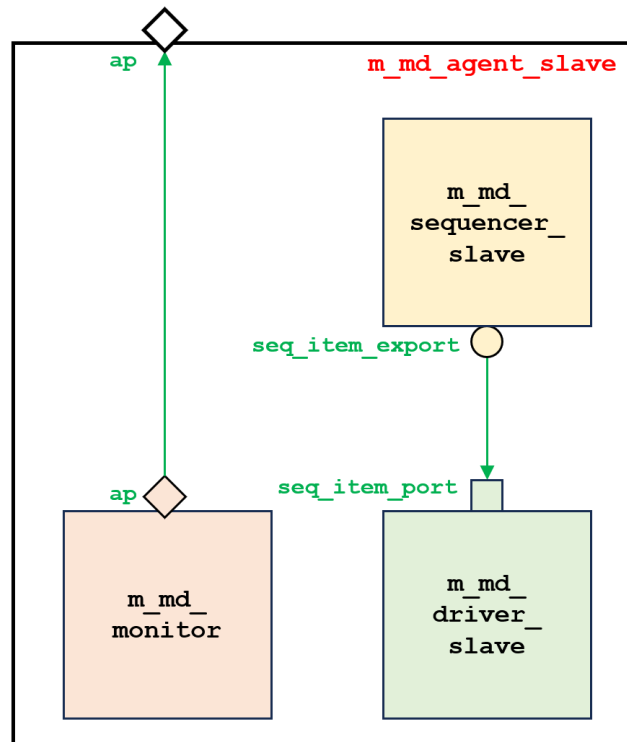


Figure 10 Structure of m_md_agent_slave



Figure 11 UML class diagram of md_agent_slave

5 Sequence Items

Various sequence items are used in this environment:

- `apb_seq_item_*`, where `*` = `base`, `drv`, or `mon`
- `md_seq_item_*`, where `*` = `base`, `drv_base`, `drv_master`, `drv_slave`, or `mon`

They are described in this chapter.

5.1 APB Sequence Items

[Figure 12](#) shows the UML class diagrams of `apb_seq_item_*`.

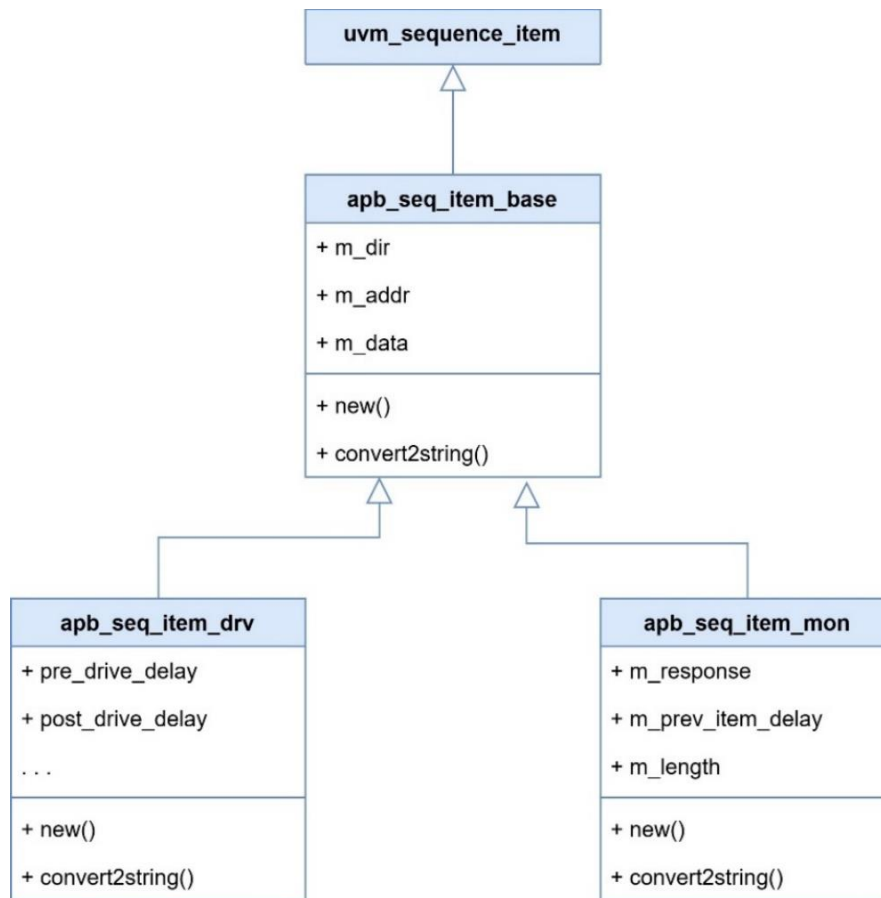


Figure 12 UML class diagrams of `apb_seq_item_*`

5.2 MD Sequence Items

[Figure 13](#) shows the UML class diagrams of `md_seq_item_*`.

[Figure 14](#), [Figure 15](#), and [Figure 16](#) show waveform diagrams that may be useful to understand the properties of `md_seq_item_drv_master`, `md_seq_item_drv_slave`, and `md_seq_item_mon`, respectively.³

³ These waveform diagrams were captured from some lectures in [Design Verification with SystemVerilog/UVM](#).

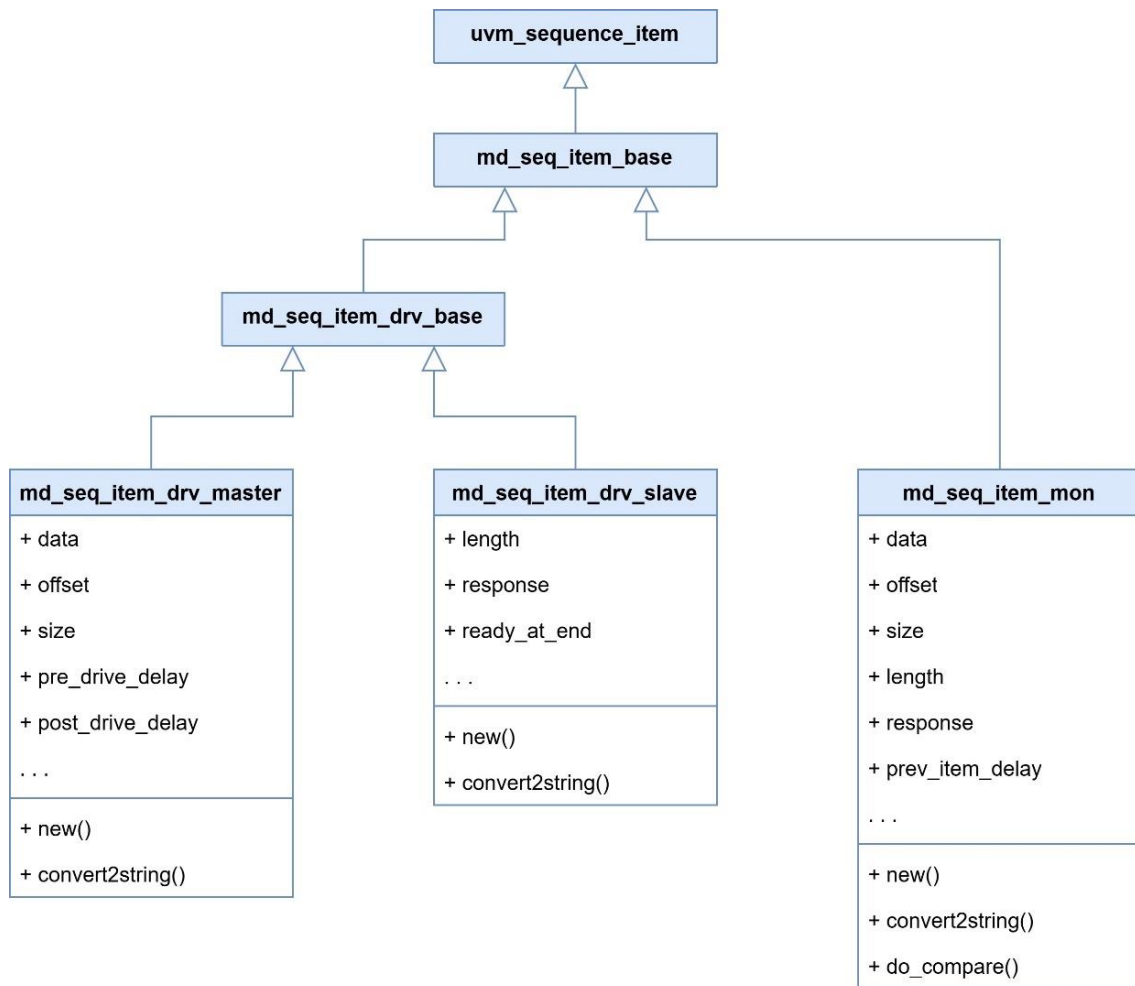


Figure 13 UML class diagrams of md_seq_item_*

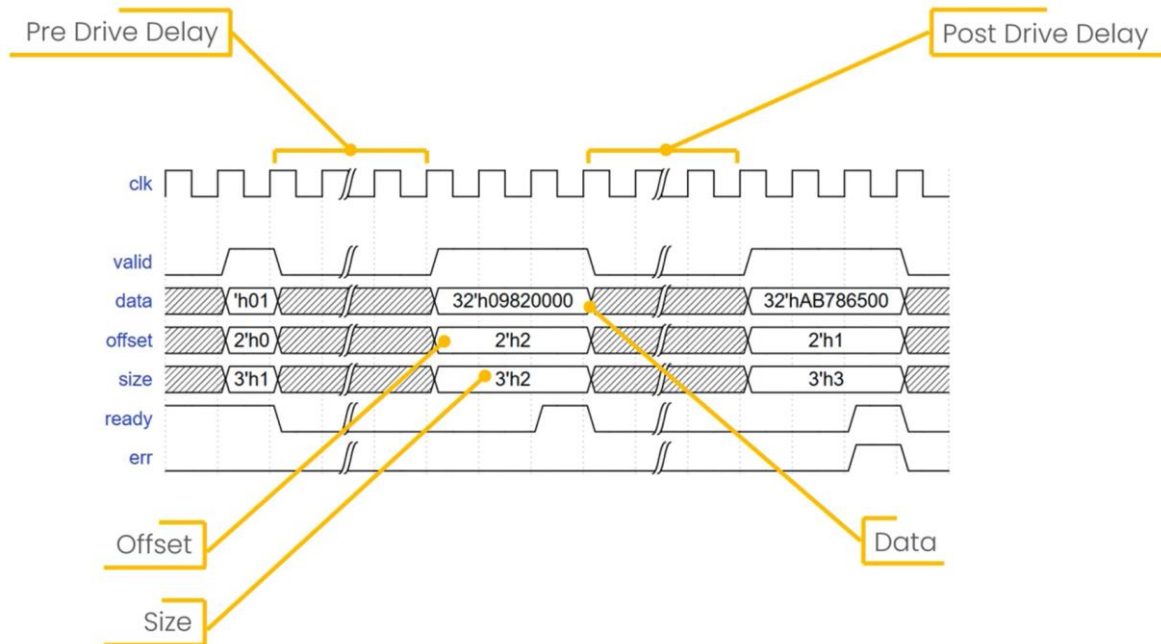


Figure 14 Driving the Rx interface using md_seq_item_drv_master

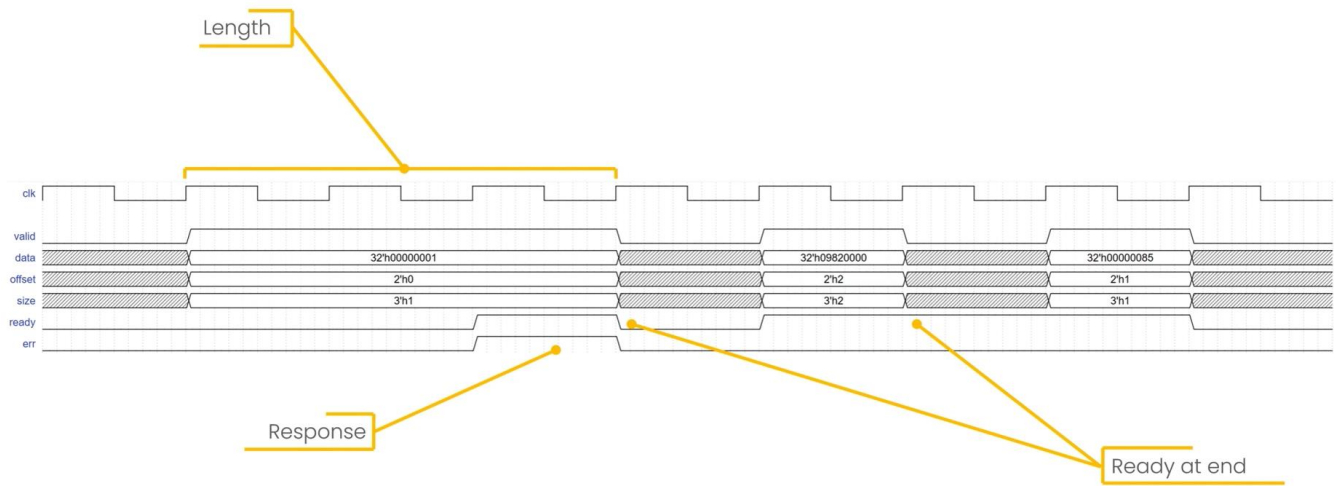


Figure 15 Driving the Tx interface using `md_seq_item_drv_slave`

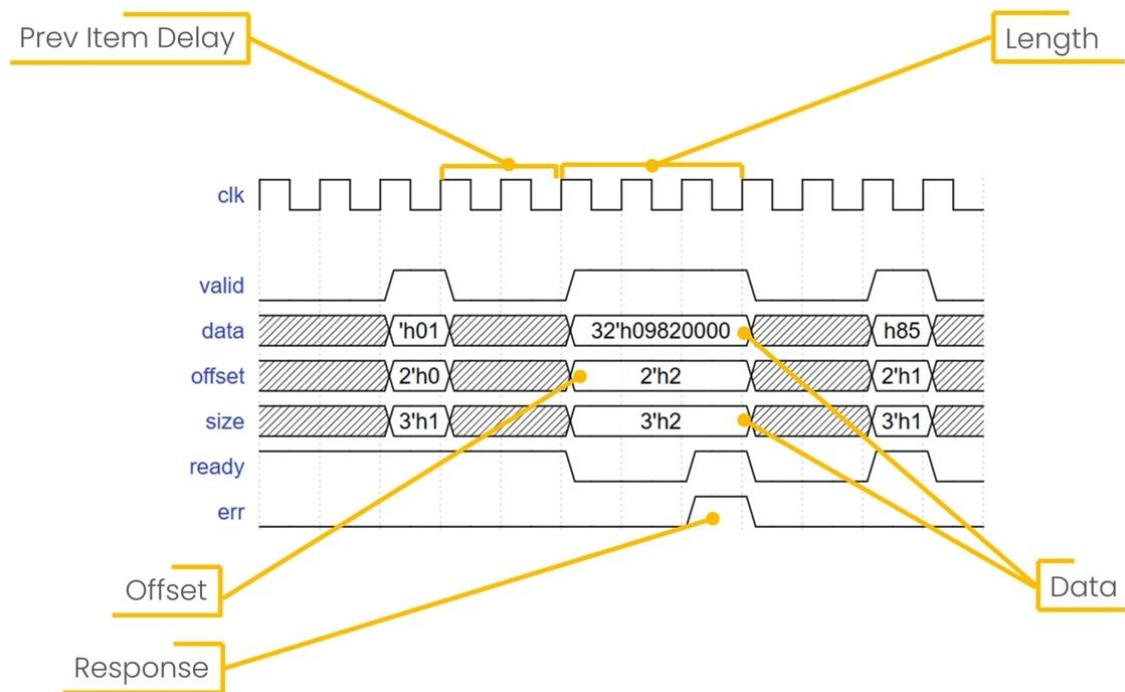


Figure 16 Monitoring the Rx and Tx interfaces using `md_seq_item_mon`

6 The Model

[Figure 17](#) reminds the reader of the block diagram of the Data Aligner module⁴, which is modeled, in our environment, by a UVM component; `align_model`.

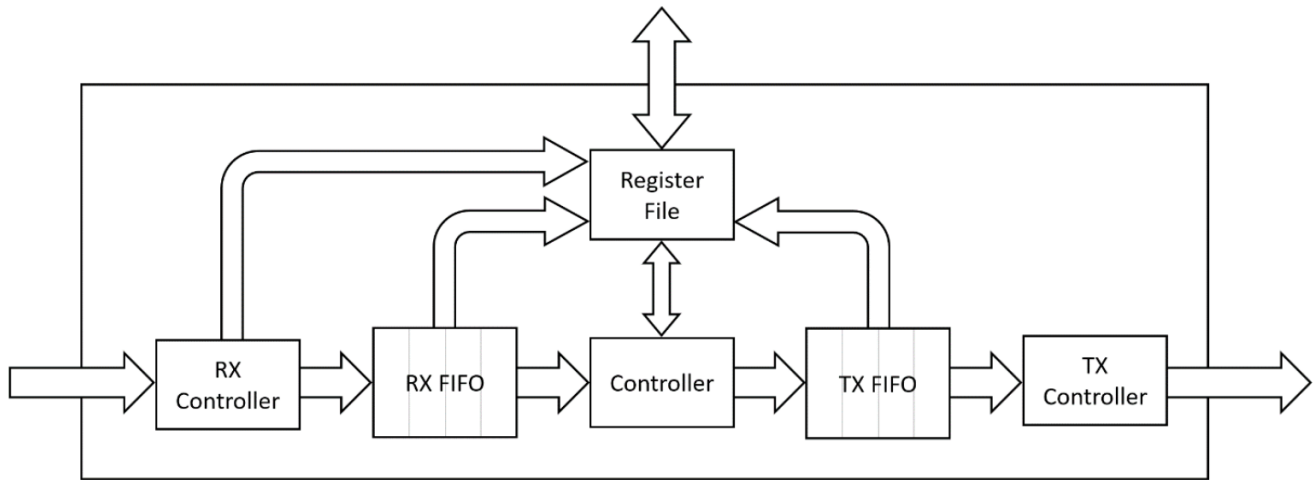


Figure 17 Block diagram of the Data Aligner module

[Figure 18](#) shows the structure of `model`, which is an instance of `align_model`.

Note: `rx_fifo` and `tx_fifo` are instances of `uvm_tlm_fifo#(md_seq_item_mon)`.

[Figure 19](#) shows the UML class diagram of `align_model`. Its properties and API are categorized in the figure.

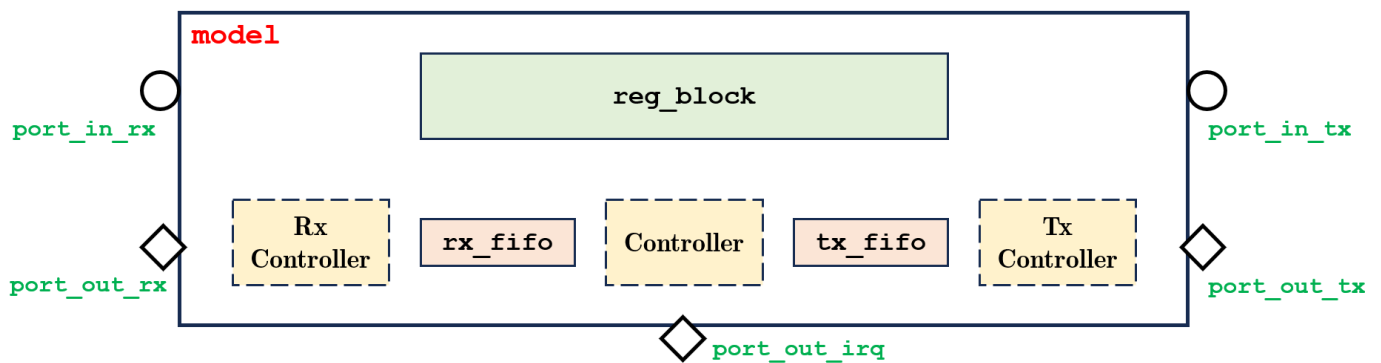


Figure 18 Structure of `model`

⁴ This block diagram is captured from *Aligner Datasheet - v1.1*.

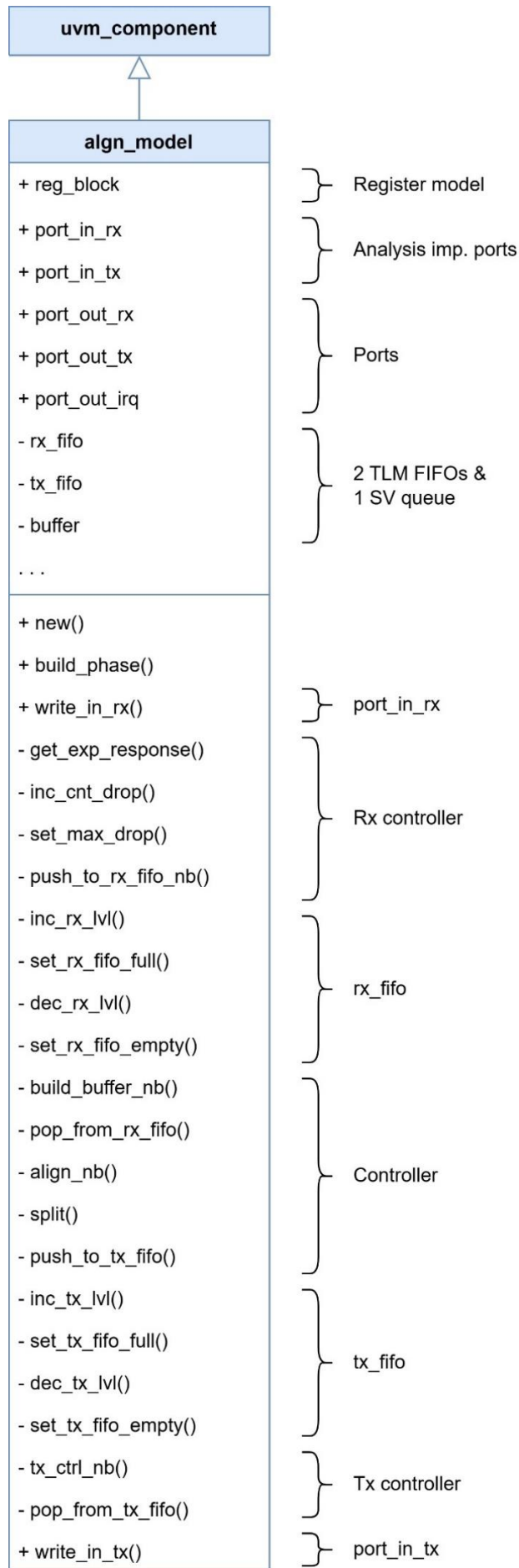


Figure 19 UML class diagram of `align_model`

6.1 The Register Model

The Data Aligner has several control and status registers accessible through the APB interface. These registers and their fields are shown in [Figure 20](#).⁵

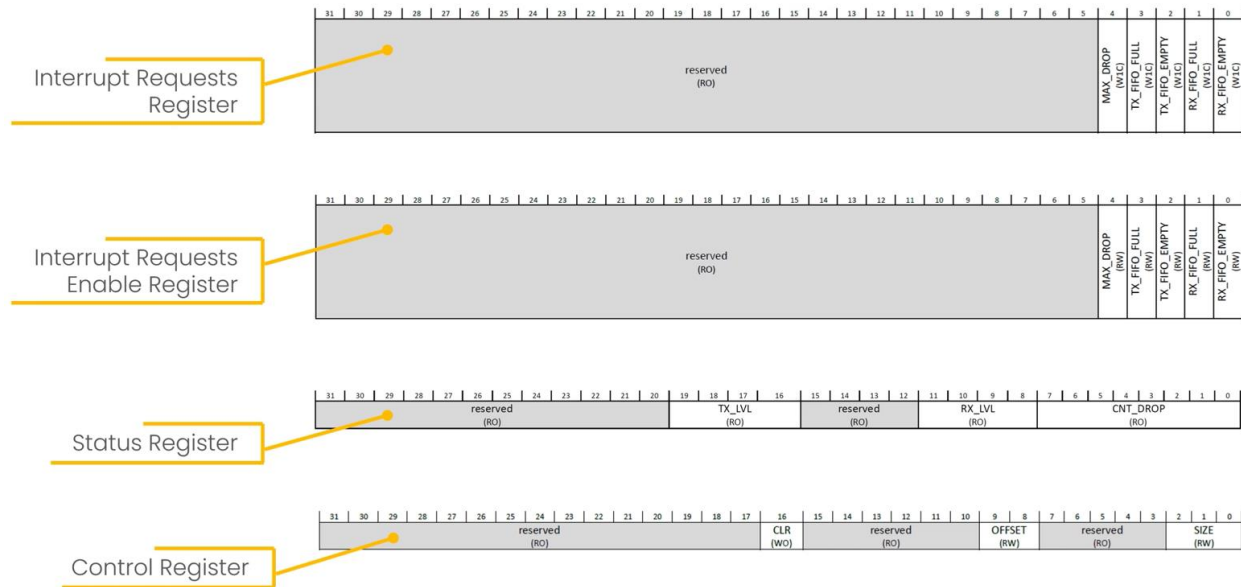


Figure 20 Registers of the Data Aligner module

The register model in model is `reg_block`, which is an instance of `align_reg_block`.

[Figure 21](#) shows the UML class diagrams of `align_reg_block` and the classes of which it is *composed*; `align_reg_ctrl`, `align_reg_status`, `align_reg_irqen`, `align_reg_irq` (registers), and `uvm_reg_map`, (address map).

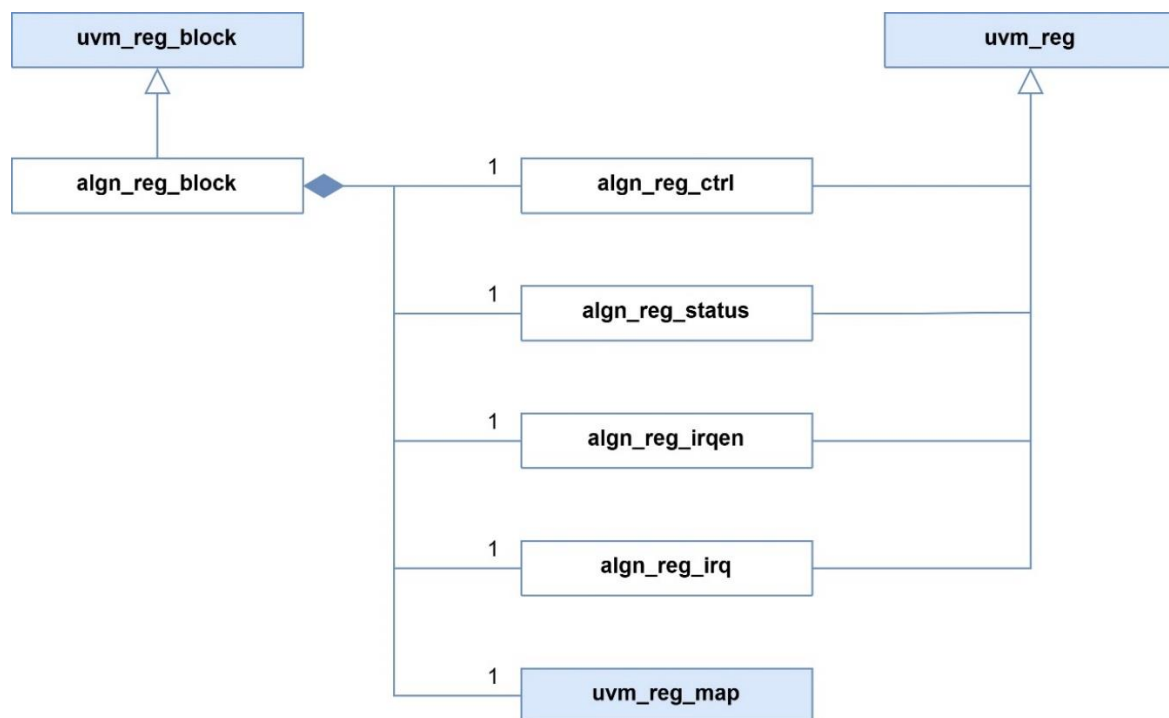


Figure 21 UML class diagrams of `align_reg_block` and the classes of which it is composed

⁵ This figure were captured from a lecture in [Design Verification with SystemVerilog/UVM](#).

7 The Scoreboard

Figure 22 shows the structure of scoreboard, which is an instance of `align_scoreboard`.

Note: `expected_*_transactions_fifo` and `actual_*_transactions_fifo` are instances of `uvm_tlm_analysis_fifo#(...)`.

Figure 23 shows the UML class diagram of `align_scoreboard` which extends `uvm_scoreboard`.

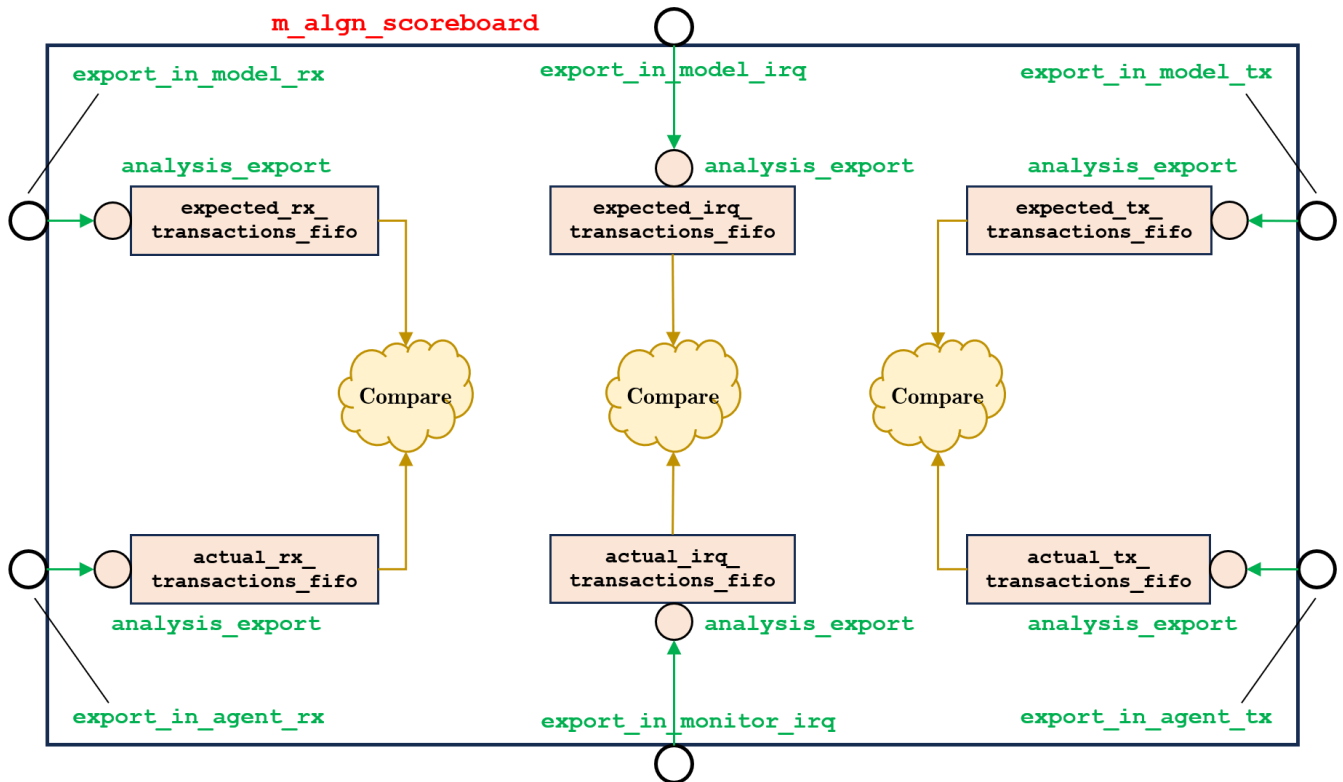


Figure 22 Structure of scoreboard

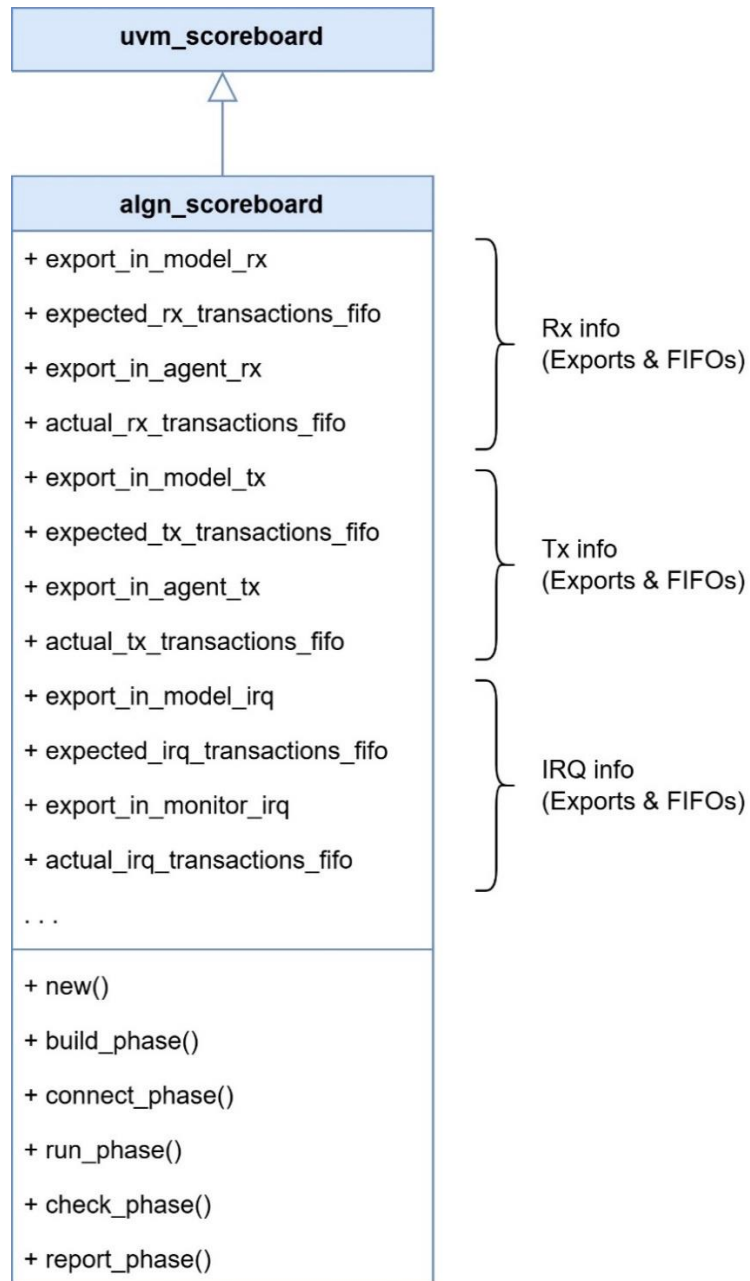


Figure 23 UML class diagram of **algn_scoreboard**

8 Sequences and Tests

Various sequences and tests are used to test the DUT. They are listed in this chapter.⁶

8.1 Sequences

Figure 24 shows the UML class diagrams of `md_sequence_*` and `align_sequence_*`.

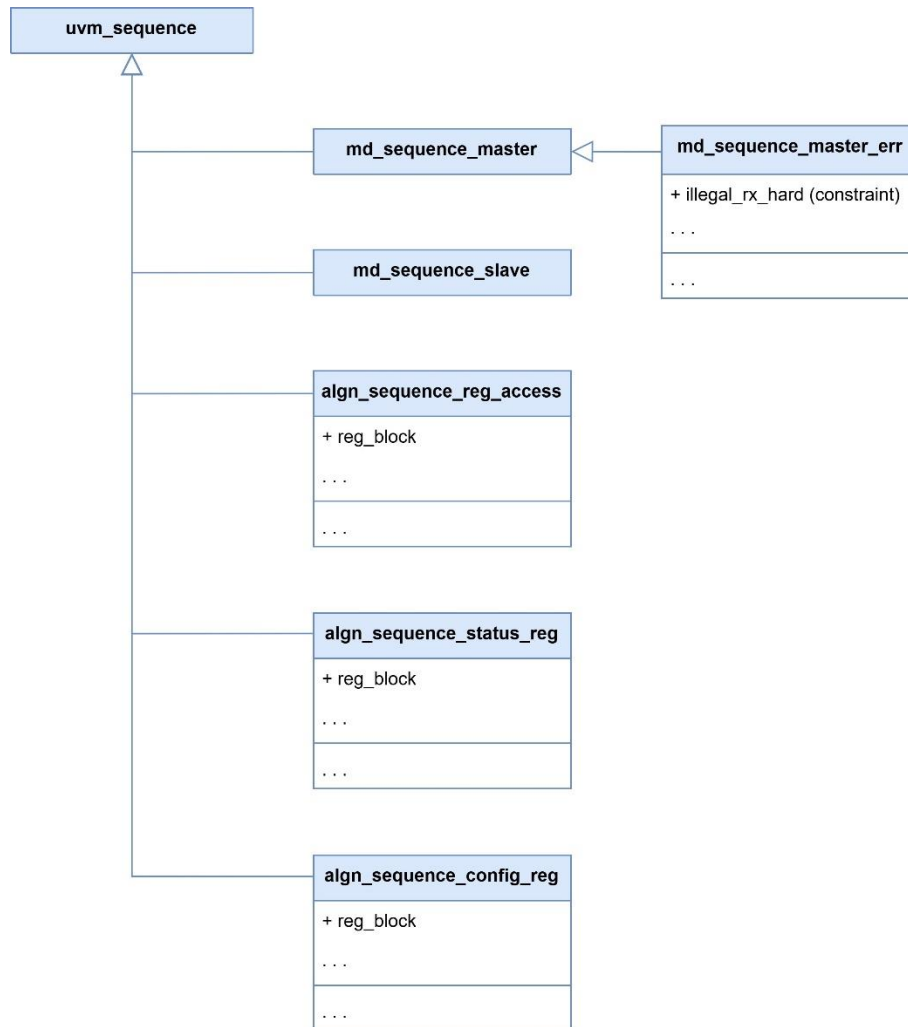


Figure 24 UML class diagrams of `md_sequence_*` and `align_sequence_*`

Notes:

- `md_sequence_master_err` inherits properties and methods from `md_sequence_master` and adds a hard constraint, `illegal_rx_hard`, to ensure that the produced sequence items are illegal.
- `align_sequence_*` need a handle to the register model (`reg_block`).

⁶ For more details about these sequences and tests, see the test plan in *Data Aligner - Verification Plan*.

8.2 Tests

Figure 25 shows the UML class diagrams of `align_test_*`.

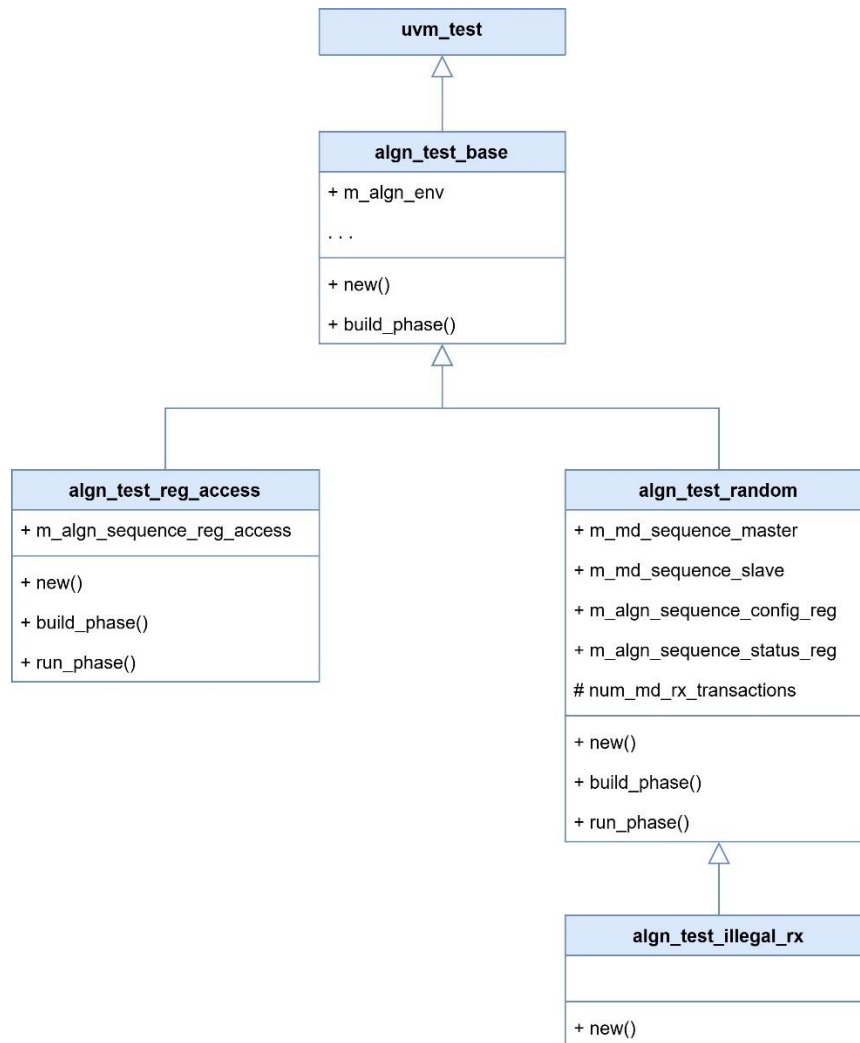


Figure 25 UML class diagrams of `align_test_*`

Note:

`align_test_illegal_rx`, which extends `align_test_random`, uses the UVM Factory to override `m_md_sequence_master` with an instance of `md_sequence_master_err`:

```
md_sequence_master::type_id::set_type_override(md_sequence_master_err::get_type());
```

9 Future Improvements

Some improvements and enhancements shall be added to this testbench for more complete verification. For example:

- Modeling, testing and checking of `pslverr`.
- Modeling, testing and checking of `reset_n`.
- Enhancing the model by making it synchronous with the DUT.
- Merging all coverage collectors in one coverage collector.
- Adding more constrained tests.